STQA IV

TY BSC IT Mumbai University

Software Verification and Validation (V&V)

- **Software Verification and Validation (V&V)** is a process that ensures software meets its **specified requirements** and functions **correctly and reliably**. It is a critical part of **software quality assurance (SQA)** and helps in **detecting defects early** in the development lifecycle.

1. Software Verification

Definition:

Verification ensures that the software is being developed **correctly** according to the **specifications and design documents**. It answers the question:

✅ **"Are we building the product right?"**

Key Characteristics of Verification:

- Focuses on **static** methods (i.e., without executing the code).
- Checks **documents, design, architecture, and requirements**.
- Performed in the **early phases** of development.

Common Verification Techniques:

- 🔹 **Reviews** – Formal or informal checks of documents and code.
- 🔹 **Walkthroughs** – Step-by-step code or design review.
- 🔹 **Inspections** – Detailed analysis of artifacts to find defects.
- 🔹 **Static Analysis** – Automated tools analyze ↓ source code.

2. Software Validation

Definition:

Validation ensures that the **final product meets user needs** and performs its intended functions. It answers the question:

✅ **"Are we building the right product?"**

Key Characteristics of Validation:

- Focuses on **dynamic** testing (i.e., executing the software).
- Checks whether the software works **as expected in real scenarios**.
- Performed in the **later phases** of development.

Common Validation Techniques:

- **Unit Testing** – Tests individual components.
- **Integration Testing** – Tests interactions between components.
- **System Testing** – Ensures the complete system meets requirements.
- **User Acceptance Testing (UAT)** – Confirms software is ready for deployment.

Key Differences: Verification vs. Validation

Feature	Verification	Validation
Focus	Ensures the software is built **correctly**	Ensures the **correct** software is built
Type	**Static** (no execution)	**Dynamic** (execution-based)
Activities	Reviews, inspections, walkthroughs	Testing (unit, integration, system, UAT)
Timing	Early development phases	Later development phases
Goal	Compliance with requirements	Meeting user needs

Verification Workbench

- The **Verification Workbench** is a structured framework used to systematically verify that a software product meets its **requirements and design specifications**. It consists of a set of **tools, techniques, and processes** to ensure that software is being developed **correctly** before it moves to the validation phase.

Key Components of the Verification Workbench

Component	Description
Requirements Review	Ensures that the software requirements are clear, complete, and feasible.
Design Verification	Evaluates if the software design correctly implements the requirements.
Code Review	Systematic examination of source code to identify errors.
Static Analysis	Uses automated tools to detect code vulnerabilities and inconsistencies.
Walkthroughs	Step-by-step analysis of documents and code by team members.
Formal Inspections	Rigorous peer reviews to detect defects in early phases.
Traceability Analysis	Ensures that all requirements are correctly mapped to design, code, and test cases.

Process of Using the Verification Workbench

1. **Planning** – Define verification objectives, scope, and methods.
2. **Review & Inspection** – Perform systematic checks of requirements, design, and code.
3. **Static Analysis** – Use tools to detect defects without executing the software.
4. **Traceability Mapping** – Ensure all requirements are covered in design and testing.
5. **Issue Reporting** – Document defects and track their resolution.
6. **Verification Report** – Summarize findings and improvements before moving to validation.

Methods of Verification in Software Testing

- **Software Verification** ensures that the software is built **correctly** by checking whether it meets **design specifications, coding standards, and requirements** before execution. It is a **static process** that does not involve running the software.

1. Reviews

✅ Conducted at different stages (Requirements Review, Design Review, Code Review).

✅ Helps identify issues **before coding starts**.

2. Walkthroughs

✅ The author explains the document or code to a group.

✅ Useful for **knowledge sharing** and early defect detection.

3. Inspections

✅ **More formal than walkthroughs**, involves a moderator, checklist, and defect log.

✅ Detects issues in **early development stages**, saving time and cost.

4. Static Analysis

- ✅ Uses tools like **SonarQube, Coverity, Lint** to find code issues.
- ✅ Finds **security vulnerabilities, syntax errors, and dead code**.

5. Formal Methods

- ✅ Uses **mathematical proofs and logic** to verify software correctness.
- ✅ Applied in **highly critical systems** like aerospace and healthcare.

6. Prototyping

- ✅ A **basic working model** is created to validate requirements.
- ✅ Helps in **reducing requirement misunderstandings**.

7. Traceability Analysis

- ✅ Ensures all requirements are correctly implemented.
- ✅ Uses **traceability matrices** to track the mapping between requirements and test cases.

Types of Reviews Based on Stage/Phase

1. **Requirements Review** (Early Stage)

- ✅ Ensures requirements are **clear, complete, and feasible**
- ✅ Detects **ambiguities, inconsistencies, and missing requirements**
- ✅ Conducted **before design begins**

2. **Design Review** (Before Development)

- ✅ Evaluates **software architecture and design documents**
- ✅ Ensures design meets **functional and non-functional requirements**
- ✅ Checks for **scalability, security, and performance**

3. **Code Review** (During Development)

- ✅ Examines **source code for correctness, readability, and efficiency**
- ✅ Detects **logical errors, security vulnerabilities, and coding standard violations**
- ✅ Can be **manual (peer reviews) or automated (static analysis tools)**

4. **Test Plan Review** (Before Testing)

- ✅ Ensures **test cases cover all functional and boundary conditions**
- ✅ Checks alignment with **requirements and business logic**
- ✅ Identifies **gaps in test strategy**

5. **User Acceptance Review** (Before Deployment)

- ✅ Confirms that the software **meets business and user needs**
- ✅ Conducted with **end-users or stakeholders**
- ✅ Determines **if the system is ready for production**

Entities Involved in Software Verification

Key Entities Involved in Verification:

Entity	Role in Verification
Business Analyst (BA)	Ensures requirements are complete, clear, and testable. Participates in requirement reviews.
Project Manager (PM)	Oversees verification activities, ensures adherence to quality standards, and manages risks.
Software Architect	Reviews system and software design to ensure correctness and alignment with requirements.
Developers	Participate in **code reviews, static analysis, and walkthroughs** to verify correctness before testing.
Testers/QA Engineers	Conduct reviews of test plans, test cases, and perform static testing to ensure coverage.
Configuration Manager	Ensures that the correct software versions and configurations are verified.
Technical Writers	Review documentatic cluding user manuals and design documents, for completeness and accuracy.

Reviews in the Software Testing Lifecycle

- **Reviews** in the software testing lifecycle are **systematic evaluations** of software artifacts to identify defects **early** in the development process. These reviews help improve **software quality, reduce rework, and ensure compliance with requirements** before formal testing begins

Types of Reviews in the Testing Lifecycle

Review Type	Purpose	Conducted By	Stage in SDLC
Requirement Review	Ensures requirements are clear, complete, and testable.	Business Analysts, Developers, Testers, Stakeholders	**Requirement Analysis**
Test Plan Review	Checks if the test plan covers all testing aspects (scope, strategy, resources).	Test Manager, QA Team, Developers	**Test Planning**
Test Case Review	Ensures test cases cover all functional and boundary conditions.	Testers, Developers, QA Leads	**Test Design**
Code Review	Examines source code for errors, security, and best practices.	Developers, Peers, QA Team	**Development Phase**
Design Review	Evaluates system architecture, data flow, and integration points.	Architects, Developers, Testers	**Design Phase**

Test Execution Review	Analyzes test results, logs, and defect reports for completeness and accuracy.	Testers, Test Leads, QA Manager	**Test Execution**
User Acceptance Review	Ensures the software meets business needs before deployment.	Clients, End Users, Business Analysts	**Pre-Deployment**

Coverage in Verification

- **Coverage in Verification** refers to the extent to which software **artifacts (requirements, design, and code)** are checked against specified standards to ensure correctness before execution.
- It ensures that **all aspects of the software are verified** to detect defects early in the development lifecycle.

Types of Coverage in Verification

Coverage Type	Description	Purpose
Requirement Coverage	Ensures all software requirements are reviewed and mapped to design/code/test cases.	Avoids missing functionalities.
Design Coverage	Verifies that all system architecture and component designs meet the specifications.	Ensures correct implementation of system structure.
Code Coverage	Analyzes source code to ensure all parts are reviewed and follow coding standards.	Detects logical errors and dead code.
Test Coverage	Measures how well the test cases cover functional and non-functional aspects.	Ensures sufficient validation through test cases.
Traceability Coverage	Maps requirements → design → code → test cases → defects.	Ensures completeness and alignment of all artifacts.

Concerns of Verification in Software Testing

- Software **Verification** ensures that the software product is developed **correctly** as per the specifications. However, several concerns can impact the effectiveness of verification.

Key Concerns in Software Verification

Concern	Description	Impact
Incomplete Requirements	If requirements are ambiguous or missing, verification cannot ensure correctness.	Leads to defects in later stages.
Poor Documentation	Inadequate design and requirement documents make verification difficult.	Causes misunderstandings in development and testing.
Lack of Testability	Some requirements may be difficult to verify due to vague or subjective definitions.	Reduces verification accuracy.
Insufficient Reviews	Incomplete or skipped requirement/design/code reviews may result in undetected defects.	Increases bug leakage to later stages.
Inadequate Traceability	If requirements are not mapped to design and test cases, verification coverage is reduced.	Missing features or test cases.
Static Analysis Limitations	Automated tools might not detect logical or domain-specific issues.	False positives or missed defects.

Software Validation

- **Software Validation** is the process of ensuring that the final software product meets the **intended requirements** and functions as expected in a real-world environment. It answers the question:
- ✅ **"Are we building the right product?"**
- Unlike **verification**, which checks if the product is being built **correctly**, validation ensures that the **end product meets user needs** and performs its intended function

Methods of Software Validation

Validation Method	Purpose
Unit Testing	Tests individual components for correctness.
Integration Testing	Checks interactions between components.
System Testing	Evaluates the entire system's behavior.
User Acceptance Testing (UAT)	Confirms software meets business/user requirements.
Performance Testing	Ensures the system can handle expected loads.
Security Testing	Checks for vulnerabilities and data protection.

Difference Between Verification & Validation

Aspect	Verification	Validation
Question Answered	"Are we building the product right?"	"Are we building the right product?"
Focus	Process-oriented	Product-oriented
Type	Static (Reviews, Inspections)	Dynamic (Testing, Execution)
Performed By	Developers, QA, Reviewers	Testers, End Users, Stakeholders
Goal	Ensure compliance with specifications	Ensure real-world usability

Validation Workbench

- In software testing, the term "Validation Workbench" refers to specialized tools or frameworks designed to facilitate the validation process, ensuring that software products meet their intended requirements and function as expected.
- These tools assist testers in structuring, executing, and monitoring validation activities across various testing phases.

Key Examples of Validation Workbenches:

- **Security Weaver's Validation Workbench**: This tool streamlines the testing of SAP roles and authorizations by automating the creation of test environments. It generates test user accounts and assigns necessary roles, enhancing communication between administrators and testers. This automation reduces the time and complexity associated with authorization testing.
- **Appvance's Validation Workbench**: As a submodule within the Appvance platform, this tool enables the creation and modification of structured and personalized validations. It allows for automatic verification of test conditions, contributing to efficient test design and execution.

Levels of Validation

•**Unit Testing**: This level involves testing individual components or modules of the software to ensure they function correctly in isolation. It verifies that each unit performs as designed.

•**Integration Testing**: At this stage, multiple units are combined and tested as a group to identify issues in their interactions. The goal is to detect interface defects between modules.

•**System Testing**: This comprehensive testing evaluates the complete and integrated software product. It assesses the system's compliance with specified requirements.

•**Acceptance Testing**: Conducted to determine whether the system meets the acceptance criteria and is ready for deployment. This testing is often performed by end-users or clients to validate the software's functionality in real-world scenarios.

- Additionally, the validation process can be broken down into specific stages to ensure thorough evaluation:
- **Design Qualification (DQ)**: Involves creating a list of end-user business requirements and designing a validation testing plan to address them before launching the product.
- **Installation Qualification (IQ)**: Ensures that the software is installed correctly and that the test environment matches the intended production environment.
- **Operational Qualification (OQ)**: Involves testing the product with various operations to ensure it meets specified user requirements. This includes unit testing, integration testing, and system testing.
- **Performance Qualification (PQ)**: Verifies that the product performs according to business needs in real-world conditions. This stage often includes alpha and beta testing.

Coverage in validation

- In software testing, **test coverage** is a critical metric that measures the extent to which the application's code, functionalities, or features are exercised by the test cases.
- It provides insight into which parts of the application have been tested and identifies areas that may require additional attention.

Types of Test Coverage:

•**Product Coverage**: This technique assesses various parts or modules of the application to ensure comprehensive testing. For example, in a shopping cart application, product coverage would involve testing functionalities like adding or removing items, handling maximum item limits, and managing out-of-stock scenarios.

•**Risk Coverage**: Focuses on identifying and thoroughly testing high-risk elements within the application. For instance, in an e-Commerce app, a critical risk element could be the integration with third-party payment gateways. Testing would involve scenarios such as successful payment processing, handling payment failures, and ensuring secure data transmission.

•**Requirements Coverage**: Ensures that the application meets all specified customer requirements. This involves verifying that each feature and functionality aligns with the documented expectations and performs as intended.

- **Code Coverage**: Measures the extent to which the source code has been tested. It includes various metrics such as:
  - **Function Coverage**: Determines whether each function in the code has been invoked during testing.
  - **Statement Coverage**: Assesses whether each executable statement has been executed.
  - **Branch Coverage**: Evaluates whether all possible branches (e.g., in conditional statements) have been tested.
  - **Condition Coverage**: Checks whether all boolean expressions have been evaluated to both true and false

Management of Verification and Validation

- **Planning and Strategy:**
- **Define clear objectives:** Establish specific goals for V&V activities, aligning them with project requirements and stakeholder expectations.
- **Develop a V&V plan:** Outline the scope, methods, resources, and timelines for verification and validation activities.
- **Identify critical areas:** Prioritize V&V efforts on critical components and functionalities of the software.
- **Establish acceptance criteria:** Define clear criteria for evaluating the success of V&V activities.

- **Verification Activities:**
- **Requirements review:** Conduct thorough reviews of requirements documents to ensure clarity, completeness, and consistency.
- **Design inspections:** Inspect design documents to identify potential flaws and ensure adherence to standards.
- **Code reviews:** Review code to detect errors, ensure code quality, and enforce coding standards.
- **Testing:** Perform various types of testing, including unit testing, integration testing, and system testing, to verify that the software meets specified requirements.

- **Validation Activities:**
- **User acceptance testing (UAT):** Involve end-users in testing the software to ensure it meets their needs and expectations.
- **Beta testing:** Release the software to a limited number of users for real-world testing and feedback.
- **Usability testing:** Evaluate the user interface and user experience to ensure ease of use and effectiveness.
- **Performance testing:** Assess the software's performance under various conditions to ensure it meets performance requirements.

- **Management and Control:**
- **Traceability:** Maintain traceability between requirements, design elements, code, and test cases to ensure comprehensive coverage.
- **Configuration management:** Manage and control changes to software artifacts to maintain consistency and avoid errors.
- **Defect tracking:** Track and manage defects identified during V&V activities, prioritizing their resolution.
- **Metrics and reporting:** Collect and analyze metrics to assess the effectiveness of V&V activities and identify areas for improvement.
- **Communication:** Establish clear communication channels among stakeholders to ensure effective collaboration and timely feedback.

- **Continuous Improvement:**
- **Lessons learned:** Capture lessons learned from V&V activities to improve future processes.
- **Process improvement:** Continuously evaluate and improve V&V processes to enhance efficiency and effectiveness.
- **Tooling:** Utilize appropriate tools to support V&V activities, such as test management tools, defect tracking tools, and code analysis tools.

Software development verification and validation activities.

- **Verification Activities (Are we building the product right?):**
- **Requirements Review:**
  - **Goal:** Ensure requirements are clear, complete, consistent, and testable.
  - **Activities:**
    - Reviewing requirements documents with stakeholders.
    - Identifying ambiguities, inconsistencies, or missing information.
    - Ensuring requirements are traceable and prioritized.

- **Design Inspections: Goal:** Evaluate software design to identify potential flaws and ensure adherence to standards.
- **Activities:**
  - Inspecting design documents (architecture, modules, interfaces).
  - Checking for design flaws, inefficiencies, or security vulnerabilities.
  - Ensuring design aligns with requirements and coding standards.
- **Code Reviews: Goal:** Examine code to detect errors, ensure code quality, and enforce coding standards.
- **Activities:**
  - Manual code reviews by developers.
  - Static code analysis using automated tools.
  - Identifying potential bugs, code smells, or security issues.

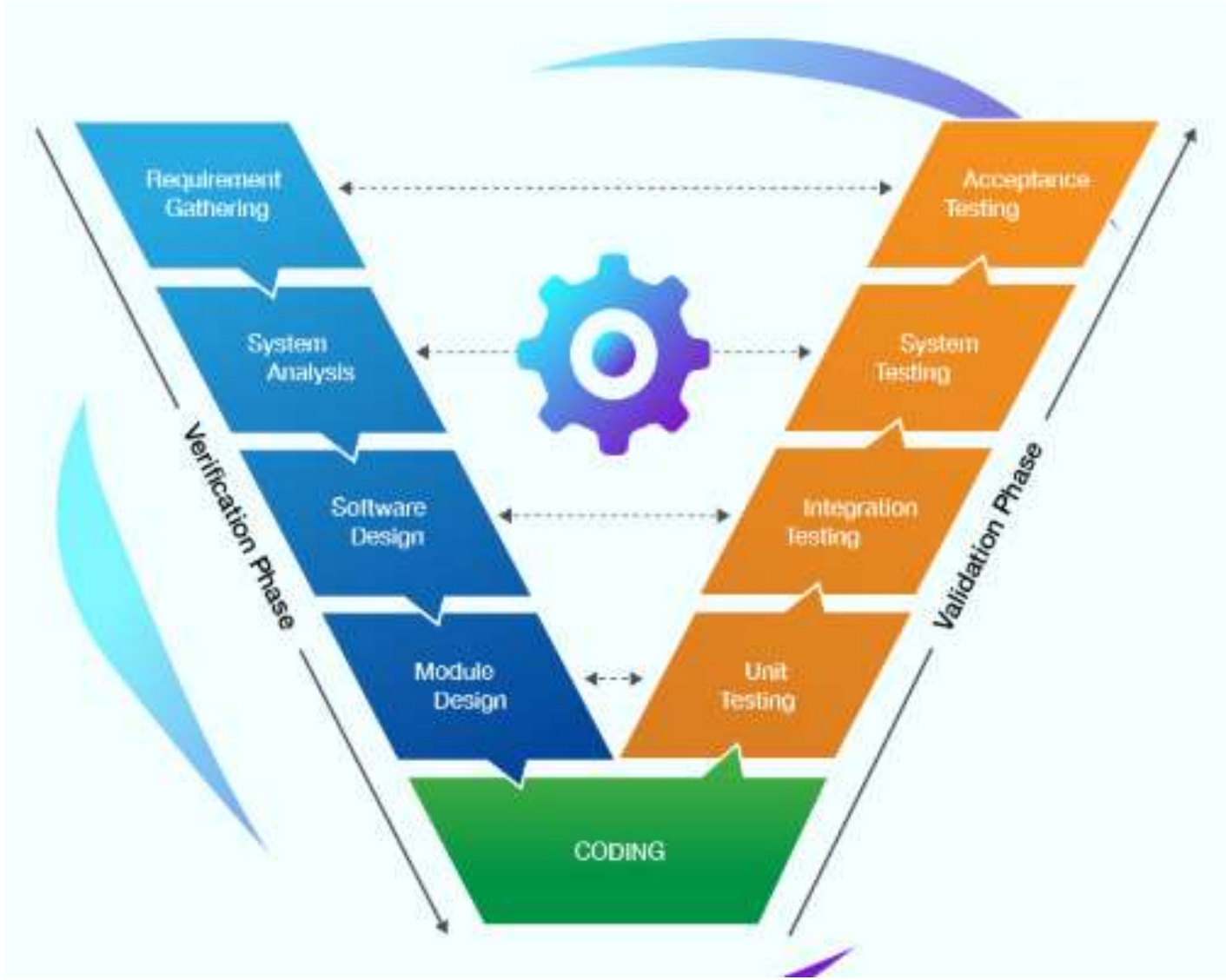
- **Testing: Goal:** Verify that the software meets specified requirements.
- **Activities:**
  - Unit testing: Testing individual components or modules.
  - Integration testing: Testing interactions between components.
  - System testing: Testing the entire software system.
  - Non-functional testing: Performance, security, usability testing.

- **Validation Activities (Are we building the right product?):**
- **User Acceptance Testing (UAT):**
  - **Goal:** Involve end-users to ensure the software meets their needs and expectations.
  - **Activities:**
    - End-users test the software in a real-world environment.
    - Providing feedback on usability, functionality, and performance.
    - Determining if the software meets user acceptance criteria.
- **Beta Testing: Goal:** Release the software to a limited number of users for real-world testing and feedback.
- **Activities:**
  - Gathering feedback from beta users on their experience.
  - Identifying any remaining bugs or usability issues.
  - Using feedback to improve the software before final release.

- **Usability Testing: Goal:** Evaluate the user interface and user experience to ensure ease of use and effectiveness.
- **Activities:**
  - Observing users interacting with the software.
  - Collecting data on user performance and satisfaction.
  - Identifying areas for improvement in the user interface
- **Performance Testing: Goal:** Assess the software's performance under various conditions.
- **Activities:**
  - Load testing: Testing the software under heavy load.
  - Stress testing: Testing the software beyond normal limits.
  - Performance testing: Measuring response times and resource usage.

What is V Model in Software Testing?

- V Model in Software testing is an SDLC model where the test execution takes place in a hierarchical manner. The execution process makes a V-shape. It is also called a Verification and Validation model that undertakes the testing process for every development phase.
- For instance, if the developers roll out a new feature. Only after the testing of that new feature is over does the next phase of the V model start.
- The V model in Software testing is useful for providing a systematic and visual representation of the SDLC process in a sequential manner. The shape ‘V’ represents the development and testing phase occurring together before moving on to the next stage.


Requirement Gathering
System Analysis
Software Design
Module Design
CODING
Unit Testing
Integration Testing
System Testing
Acceptance Testing
Verification Phase
Validation Phase

Importance of Software Model:

- A software project involves many development life cycles, so selecting the correct one becomes difficult. Software Development Models require careful selection by looking at the budget, team size, project criticality, and criticality of the product.
- Choosing the suitable model will improve the efficiency of your IT projects and manage risks associated with the software development lifecycle

Importance of Correct Model

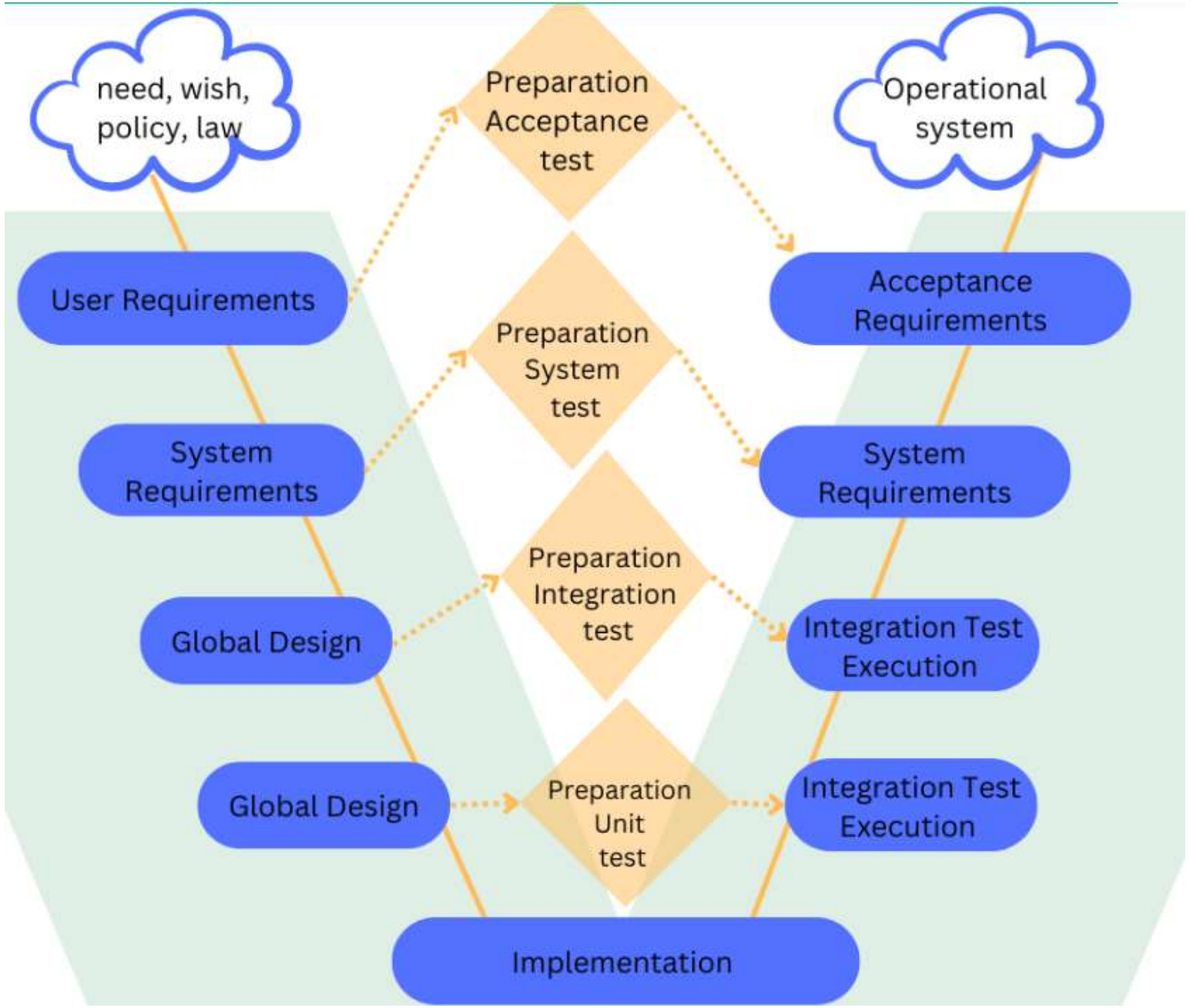
- Looking at the PIC-1 and PIC-2 models, were you able to notice anything? Yes, you are right! PIC-1 shows mistakes in house construction, and the constructed balcony is not helpful as it is practically impossible for one can go there apart from, of course, you are a Spiderman. On the other hand, PIC-2 gives us an idea of practical, well-designed construction. It followed a proper design model before construction, whereas PIC1 has not. Hence, this needs reconstruction to make it correct.
- In the case of PIC-1, the reconstruction cost is more as the issue comes into the picture late in the construction phase. Effective **construction time tracking** could have helped identify and address these issues earlier, reducing reconstruction costs and efforts significantly. We could have saved/minimized the reconstruction cost and efforts to the optimum level if a correct house model had been followed before construction or if the problem had been detected earlier in the construction phase.

The same rule applies in Software Engineering too. The necessity of correct software testing models in the Software Development Life Cycle (SDLC) is inevitable. Ideally, a Software Development Life Cycle consists of the following essential steps:

1. Planning
2. Analysis
3. Design
4. Implementation
5. Testing & Integration
6. Maintenance

V Model

- Also known as the verification and validation model, the V model guides where testing needs to begin as early as possible in the SDLC life cycle. Testing is not only an execution-based activity. It also involves various activities that must be covered before the end of the coding phase. V Model corresponds to both testing activities and development activities in parallel.
- The following diagram illustrates the different phases in a V Model of the SDLC.


need, wish, policy, law
Preparation Acceptance test
Operational system
User Requirements
Preparation System test
Acceptance Requirements
System Requirements
System Requirements
Preparation Integration test
Global Design
Integration Test Execution
Global Design
Preparation Unit test
Integration Test Execution
Implementation

- As shown above, the V Model contains Verification phases on one side and Validation phases on the other. The implementation/coding phase joins the verification and validation phases in V-shape. Thus it is called a V Model.
- Now let’s deep dive into the V Model verification and validation phases.

Verification Phases of the V Model

- **1. User Requirements / Requirement Analysis:**
- Remember the example given at the beginning about house construction; before starting the construction work, an individual needs to gather specific details such as financial options, appropriate location, right contractor for constructing a house, construction and building materials, availability of water and electricity, etc.

- Similarly, before starting any software project, the authorities would need to gather certain information, such as having a detailed discussion with customers to know and understand their exact requirements and expectations of the product.
- Analyses of the gathered requirements need to be done, then refined and come up with accurate, consistent, and unambiguous requirements, mainly in the form of documents called Requirements Specification or Requirements Definition Documents.

- This document will capture both the product’s functional and non-functional requirements and the requirement’s priority. The Requirements document is signed-off formally by the stakeholders, which becomes the basis for all further activity.
- Based on this, the user acceptance criteria test cases are developed. The customers/stockholders, Business Analysts, and Project Managers are the key members of the requirement analysis phase

- **2. System Requirements:**
- Once we have collected the clear and detailed product requirements, the next phase i.e., system requirements begins in earnest. Taking the Requirements Specification inputs in this phase next Functional Requirements Specification or **System Requirement Specification** begins.
- Business Analysts and Developer Leads take the user requirements and turn those into different features that the product should have at the end.
- During this system requirement phase, teams will revisit and finalize the complete hardware requirements, resources, and communication setup for the product under development.
- The system test plan is developed based on the system requirements phase. These requirements are segregated into different features, and features are grouped. Each feature is prioritized in this phase.

- **i. Global Design / High-Level Design:**
- The global design phase is also referred to as High-Level Design(HLD) phase. Each feature is again broken down into different modules in this phase.
- The Architects, Developers, Database designers, and other required members take the elements segregated in the system requirements phase as input and develop an overall high-level technical design.
- This high-level design document has all the required technical details, such as hardware, software, Programming language to be used, Database design, applications touch points, etc., clearly defined.
- They built the HLD so that each module is associated with a specific function. This information allows integration tests to be designed and documented during this stage.

- **ii. Detailed Design / Low-Level Design:**
- In the next stage, the Technical Specification i.e. HLD, is taken as the input. Individual developers or the team use it to develop a detailed design of their modules and their interface with other modules.
- Details about what type of data objects, how the data is to be collected, how the data can be integrated, business rules to be applied, and data storage are all documented in the detailed design Specification.
- This intricate design specification/document is also referred to as Low-Level Design. In this phase, simultaneously, unit tests can be designed on the internal low-level design of each module.

- **3. Implementation Phase / Coding Phase:**
- Programmers or Developers take the Low-Level Design Specification and start the implementation process. The coding of each module designed in the detailed design phase is taken up in the coding/implementation phase.
- The coding is performed based on the coding guidelines and standards of the specific language that is being used. The developed code goes through numerous code reviews-feedbacks, and implementation and is optimized for its best performance before the final build is checked into the source repository.

Validation Phases of the V Model

- **1. Component Test Execution/Unit Testing:**
- The component test execution phase is also referred to as component testing, unit testing, or module and program testing executed during the detailed design phases.
- This validation testing phase searches for defects in and verifies the functioning of software's different modules, programs, objects, classes, etc., separately testable.
- Most often, component testing stubs and drivers are used to replace the missing software and simulate the interface between the software components to achieve the testing in isolation from the rest of the system.

- Component testing may include different testing characteristics such as resource behavior, memory leaks, code coverage, robustness testing, etc. The detected defects are typically fixed as soon as they are found without formally recording the incidents.

- **2. Integration Test Execution/Integration Testing:**
- Primarily integration testing is carried out based on global or high-level design. The development team comes up with the technical specification and a corresponding integration for a test plan.
- The details are noted of how the modules or programs will be integrated, data that will be passed through the interfaces, and additional **integration testing** approaches such as Big Bang, Top-Down, Bottom-up, or functional incremental, etc.

- Integration testing is performed to expose the defects in the interfaces between components, interactions of different parts or systems such as an Operating System and File system, etc.
- Here the main focus is on how other modules or components' interaction works, not the functionality of the individual component.

- **3. System Test Execution/ System Testing:**
- **System testing** tests an integrated system/whole system or product to verify that it meets specified requirements. The main goal is to validate that the delivery system meets the design specifications.
- The testing team performs complete or thorough testing in this phase, including system, functional, and non-functional test cases. Its purpose is to find as many defects as possible and report them.
- System tests are generally executed in different development **test environments**. However, the test environments should correspond to the production environment as much as possible to minimize the risk of environment-specific bugs.

- **4. Acceptance Test Execution/User Acceptance Testing:**
- Acceptance test execution is associated with the requirements analysis phase. Testers review the Requirement specification and check whether the requirement has enough details to be tested, there is no ambiguity if non-functional requirements are mentioned as well, whether the requirements have been prioritized, and if product risks have been captured.
- This Test plan is designed focused on conducting tests concerning the user needs, requirements, and business processes.
- This determines whether or not the software product/system satisfies the acceptance criteria and enables the users, customers, or other authorized entities to determine whether or not to accept the system.

Principles of V Model

- The principles of the V model are based on Verification and Validation, which you can find in the above sections. Here, we discuss the clear principles of the V model in Software testing:
- **From Large to Small:** The first principle states that the testing needs to be done in a sequential process. It must start with identifying the requirements, creating a high-level, concise design, and detailing the design phases of the project.
- **Data and Process Integrity:** This principle places emphasis on working with data and processes together to implement a successful project design.
- **Scalability:** The V model undertakes the ability to accommodate any IT project despite the size, complexity, or duration.
- **Cross Reference:** This principle establishes a direct correlation between requirements and corresponding testing activity.
- **Clear Documentation:** Like in every other project, this principle shows how documentation is a requirement that needs to be done by both the developers and the support team.

Critical Roles and Responsibilities.

- **1. Project Manager:**
- **Responsibilities:**
  - Overall responsibility for V&V planning and execution.
  - Allocating resources (budget, personnel, tools) for V&V activities.
  - Monitoring V&V progress and ensuring it stays on track.
  - Communicating V&V results to stakeholders.
  - Making decisions based on V&V findings (e.g., whether to release the software).

- **2. V&V Lead/Manager:**
- **Responsibilities:**
  - Developing and maintaining the V&V plan.
  - Defining V&V strategies and methodologies.
  - Leading and coordinating V&V activities.
  - Managing the V&V team.
  - Reporting V&V status to the project manager.
- **3. Requirements Analyst:**
- **Responsibilities:**
  - Eliciting, analyzing, and documenting requirements.
  - Ensuring requirements are clear, complete, and testable.
  - Maintaining traceability between requirements and other artifacts.
  - Participating in requirements reviews.

- **4. Designers/Architects:**
- **Responsibilities:**
  - Creating software designs that meet requirements.
  - Ensuring designs are testable and verifiable.
  - Participating in design inspections.
- **5. Developers:**
- **Responsibilities:**
  - Writing high-quality code that adheres to standards.
  - Performing unit testing and integration testing.
  - Fixing defects identified during V&V activities.
  - Participating in code reviews.

- **6. Testers:**
- **Responsibilities:**
  - Developing test plans and test cases.
  - Executing tests and documenting results.
  - Identifying and reporting defects.
  - Performing various types of testing (e.g., functional, performance, security).
- **7. Quality Assurance (QA) Engineers:**
- **Responsibilities:**
  - Ensuring V&V processes are followed.
  - Monitoring quality metrics.
  - Identifying areas for improvement in V&V processes.
  - Participating in audits and reviews.

- **8. End-Users/Customers:**
- **Responsibilities:**
  - Participating in user acceptance testing (UAT).
  - Providing feedback on the software's usability and functionality.
  - Defining acceptance criteria.

Thank You